checkpoint.py

`src/moe_pruning/serialization/checkpoint.py · 406 lines · module moe_pruning.serialization.checkpoint`

What it does

Saves and loads pruned models in a custom format that never stores the original weights of removed experts. Plain `from_pretrained` cannot load these checkpoints; `load_pruned_model()` must be used.

How it works

- `full` format: retained weights, routers, all non-MoE weights and bias vectors in `model.safetensors` (tied tensors stored once), plus config, tokenizer and manifest.
- `delta` format: only the bias vectors (`replacements.safetensors`) and the manifest; the model is rebuilt by loading the base model and re-applying the recorded replacement. A few MB instead of a full model copy.
- Full loading builds the architecture without allocating weights, installs replacement modules first, then loads with strict key and shape validation and re-ties shared weights.
- Saving refuses to write any tensor that still holds a removed expert's weights; the check is adapter-specific.
- Both loaders support single-device or multi-GPU placement.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class CheckpointError`	Raised for invalid, inconsistent or unsupported checkpoints.
`deduplicate_state_dict(state)`	Keep one copy of tensors sharing storage; return (tensors, {alias: canonical}).
`layer_method(entry)`	Replacement method of a manifest layer entry (older manifests: bias).
`replaced_expert_prefixes(manifest)`	State-dict prefixes of all removed experts in a manifest.
`expected_bias_keys(manifest)`	State-dict keys that must exist: the biases of bias-replaced experts.
`parameters_on_meta()`	Create parameters on the meta device but buffers normally.
`removed_weight_violations(adapter, model, state_shapes, manifest)`	Stored tensors (by name/shape) that hold weights of experts the manifest lists as removed.
`stored_tensor_shapes(path)`	Name -> shape of every tensor in a safetensors file, without loading data.
`save_pruned_model(model, tokenizer, output_dir, manifest, experiment_config, readme, checkpoint_format)`	Write the checkpoint and verify that no replaced expert weights are stored.
`load_pruned_model(model_path, device, placement, base_model_source, load_kwargs)`	Rebuild a pruned model from a checkpoint written by `save_pruned_model`.

Connections

Uses: `moe_pruning.model.architecture`, `moe_pruning.model.placement`, `moe_pruning.models`, `moe_pruning.pruning.replacement`, `moe_pruning.utils.device`, `moe_pruning.utils.paths`

Used by: `scripts/export_model.py`, `scripts/replace_experts.py`, `scripts/train_biases.py`, `src/moe_pruning/model/loader.py`